University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Leopard Imaging IMX219 MIPI FF NANO 8.08 MP Camera

Srikanth Nanda

11. Februar 2026

1 Leopard Imaging IMX219 MIPI FF NANO 8.08 MP Camera

2 Next Steps

3 Quellen

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Leopard Imaging IMX219 MIPI FF NANO 8.08 MP Camera

Leopard Imaging IMX219 MIPI FF NANO 8.08 MP Camera I

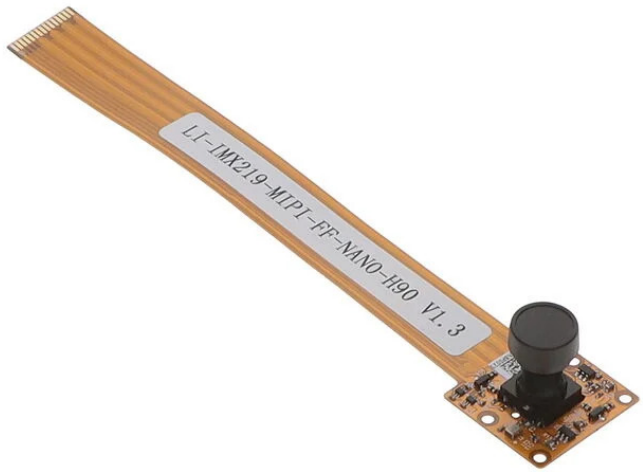


Abbildung: Leopard Imaging IMX219 MIPI FF NANO 8.08 MP Camera

Leopard Imaging IMX219 MIPI FF NANO 8.08 MP Camera I

- **Sensor**: Sony IMX219 CMOS color sensor (8.08 MP)
- **Resolution**: 3280×2464 active pixels
- **Pixel Size**: $1.12\,\mu m \times 1.12\,\mu m$
- **Optics**: Fixed-focus lens (H90 / H136 / H145 variants)
- **Field of View**: 90°, 136°, or 145° (horizontal)
- **Interface**: 2-lane MIPI CSI-2
- **Frame Rate**: Up to 21 fps at full resolution
- **Platform**: NVIDIA Jetson Nano

Hardware Connection: IMX219 to Jetson Nano I

1. Power off the Jetson Nano.
2. Connect the IMX219 camera ribbon cable to the CSI connector (CAM0).
3. Ensure correct orientation of the FPC cable (contacts facing inward).
4. Power on the Jetson Nano and verify camera detection.

Camera Indicators and Status Signals I

- **Power Indicator**: The IMX219 camera module does not include a dedicated onboard LED; power status is inferred from successful CSI initialization.
- **Activity Indication**: Camera activity is indicated at the system level by successful frame acquisition through V4L2, OpenCV, or GStreamer pipelines.
- **Driver-Level Feedback**: Kernel logs (`dmesg`) and device enumeration (`/dev/video0`) confirm sensor detection and readiness.
- **Error Indication**: Absence of video devices or capture failures indicate improper ribbon orientation, power issues, or driver misconfiguration.

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Camera Calibration Procedure I

1. Print a checkerboard calibration target.
2. Capture multiple images at different orientations.
3. Detect checkerboard corners using OpenCV.
4. Estimate intrinsic parameters and distortion coefficients.
5. Store calibration parameters for runtime correction.

IMX219 Camera Module Imports I

```
"""
@file imx219_camera_pipeline.py
@brief Complete camera bring-up, calibration, and usage pipeline for
       Leopard Imaging IMX219 MIPI FF NANO camera on Jetson Nano / Linux.

This module covers:
- Camera indicators and bring-up checks
- Frame capture
- Camera calibration using a checkerboard
- Saving and loading intrinsic parameters
- Real-time undistortion
- Example execution loop

Designed for OpenCV-based robotics and ROS 2 camera integration.
"""

import cv2
```

IMX219 Camera Module Imports II

```
18 import numpy as np
19 import glob
20 import os
21
22
23 # ============================
24 # Camera Indicators (Concept)
25 # ============================
26 #
27 # - Power LED: ON when CSI camera is powered
28 # - Activity LED: Blinks during frame capture
29 # - CSI link health: Confirmed when VideoCapture opens
       successfully
30 # - Driver readiness: Failure raises runtime exceptions
31 #
32
33
34 ##
35 # @brief Open the Leopard Imaging IMX219 camera.
36 #
```

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

IMX219 Camera Module Imports III

```
37 # Verifies CSI link, driver availability, and stream
       readiness.
38 #
39 # @param device Camera device index (default: 0)
40 # @param width Desired image width
41 # @param height Desired image height
42 # @return cv2.VideoCapture object
43 #
44 def open_camera(device=0, width=1280, height=720):
45     cap = cv2.VideoCapture(device)
46     cap.set(cv2.CAP_PROP_FRAME_WIDTH, width)
47     cap.set(cv2.CAP_PROP_FRAME_HEIGHT, height)
48
49     if not cap.isOpened():
50         raise RuntimeError(
51             "IMX219 camera not detected. "
52             "Check CSI connection, power LED, and drivers."
53         )
54     return cap
55
```

IMX219 Camera Module Imports IV

```
56
57 ##
58 # @brief Capture a single frame from the camera.
59 #
60 # @param cap Active VideoCapture object
61 # @return Captured image frame
62 #
63 def capture_frame(cap):
64     ret, frame = cap.read()
65     if not ret:
66         raise RuntimeError("Frame capture failed (CSI link
       unstable).")
67     return frame
68
69
70 ##
71 # @brief Perform camera calibration using checkerboard
       images.
72 #
```

IMX219 Camera Module Imports V

```
73 # @param image_dir Directory containing calibration images
       (*.jpg)
74 # @param checkerboard Number of inner corners (columns, rows
       )
75 # @param square_size Checker square size in meters
76 # @return (camera_matrix, distortion_coefficients)
77 #
78 def calibrate_camera(
79     image_dir,
80     checkerboard=(9, 6),
81     square_size=0.025
82 ):
83     objp = np.zeros((checkerboard[0] * checkerboard[1], 3),
       np.float32)
84     objp[:, :2] = np.mgrid[
85         0:checkerboard[0],
86         0:checkerboard[1]
87     ].T.reshape(-1, 2)
88     objp *= square_size
89
```

IMX219 Camera Module Imports VI

```
90      objpoints = []
91      imgpoints = []
92
93      images = glob.glob(os.path.join(image_dir, "*.jpg"))
94      if not images:
95          raise RuntimeError("No calibration images found.")
96
97      for fname in images:
98          img = cv2.imread(fname)
99          gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
100         ret, corners = cv2.findChessboardCorners(gray,
    checkerboard, None)
101         if ret:
102             objpoints.append(objp)
103             imgpoints.append(corners)
104
105     ret, mtx, dist, _, _ = cv2.calibrateCamera(
106         objpoints,
107         imgpoints,
108         gray.shape[::-1],
```

IMX219 Camera Module Imports VII

```
109             None,
110             None
111         )
112
113         np.savez("camera_calibration.npz", mtx=mtx, dist=dist)
114         return mtx, dist
115
116
117 ##
118 # @brief Load saved camera calibration parameters.
119 #
120 # @param filename Calibration file path
121 # @return (camera_matrix, distortion_coefficients)
122 #
123 def load_calibration(filename="camera_calibration.npz"):
124     if not os.path.exists(filename):
125         raise RuntimeError("Calibration file not found.")
126     data = np.load(filename)
127     return data["mtx"], data["dist"]
128
```

IMX219 Camera Module Imports VIII

```
129
130 ##
131 # @brief Undistort an image using camera intrinsics.
132 #
133 # @param frame Distorted input frame
134 # @param mtx Camera matrix
135 # @param dist Distortion coefficients
136 # @return Undistorted frame
137 #
138 def undistort_frame(frame, mtx, dist):
139     return cv2.undistort(frame, mtx, dist)
140
141
142 ##
143 # @brief Demonstration loop for IMX219 camera usage.
144 #
145 # - Captures frames
146 # - Applies undistortion
147 # - Displays live output
148 #
```

IMX219 Camera Module Imports IX

```
149 # Press 'q' to exit.
150 #
151 def main():
152     cap = open_camera()
153     mtx, dist = load_calibration()
154 
155     while True:
156         frame = capture_frame(cap)
157         corrected = undistort_frame(frame, mtx, dist)
158 
159         cv2.imshow("IMX219 Undistorted Output", corrected)
160 
161         if cv2.waitKey(1) & 0xFF == ord('q'):
162             break
163 
164     cap.release()
165     cv2.destroyAllWindows()
166 
167 
168 if __name__ == "__main__":
```

IMX219 Camera Module Imports X

```
169     main()
```

Listing 1: Module description and imports (IMX219Cam.py)

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Next Steps

Next Steps I

- Integrate the IMX219 camera pipeline into a ROS 2 node using `rclpy`
- Publish image frames as `sensor_msgs/Image` topics
- Add `camera_info` publishing using calibration parameters
- Enable hardware-accelerated pipelines using GStreamer on Jetson
- Perform extrinsic calibration for multi-sensor setups
- Connect the camera feed to Nav2 perception or SLAM pipelines
- Containerize the camera stack using Docker for reproducibility

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Quellen

Quellen I

Nachfolgend werden die Quellen der Bilder angegeben, die für diese Präsentation in ihrer ursprünglichen Form oder modifiziert verwendet worden sind.


Arens, Tilo u. a. (2015). *Mathematik*. 3. [pdf] http://www.gbv.de/dms/weimar/toc/551047283_toc.pdf [Link]. Heidelberg: Spektrum akademischer Verlag.


Babovsky, Hans und Werner Neundorf (2011). *Numerische Approximation von Funktionen*. Techn. Ber. TU Ilmenau. URL: https://www.tu-ilmenau.de/fileadmin/media/num/neundorf/Dokumente/Preprints/NumApp1.pdf (besucht am 13. 01. 2017).


Bonami, Pierre, Alberto Olivares und Ernesto Staffetti (2014). "Energy-Optimal Multi-Goal Motion Planning for Planar Robot Manipulators". In: *Journal of Optimization Theory and Applications* 163.1, S. 80–104. ISSN: 1573-2878. DOI: 10.1007/s10957-013-0516-0. URL: http://dx.doi.org/10.1007/s10957-013-0516-0.

Quellen II


Bundesministerium für Wirtschaft und Energie (BMWi), Hrsg. (2015). *Bekanntmachung Digitale Technologien für die Wirtschaft (PAiCE). Platforms—Additive Manufacturing—Imaging—Communication—Engineering Ein Technologiewettbewerb des Bundesministeriums für Wirtschaft und Energie*. URL: http://bmwi.de/BMWi/Redaktion/PDF/Publikationen/paice-digitale-technologien-fuer-die-wirtschaft-bekanntmachung,property=pdf,bereich=bmwi2012,sprache=de,rwb=true.pdf (besucht am 20. 07. 2016).


DIN Deutsches Institut für Normung e.V. (Sep. 1988). *DIN 66025-2:1988-09: Programmaufbau für numerisch gesteuerte Arbeitsmaschinen: Wegbedingungen und Zusatzfunktionen*. Norm. Berlin.


Farin, G. (1994). *Kurven und Flächen im Computer Aided Geometric Design. Eine praktische Einführung*. Vieweg.

Quellen III

- Farin, G. und J. Hoschek (2002). *Handbook of Computer Aided Geometric Design*. Elsevier Science.
- Farin, Gerald (2002). *Curves and Surfaces for CAGD*. 5. [pdf]. San Diego, CA: Academic Press.
- Farouki, Rida und Jyothirmai Srinathu (Jan. 2017). "A real-time CNC interpolator algorithm for trimming and filling planar offset curves". In: *Computer-Aided Design* 86. DOI: 10.1016/j.cad.2017.01.001.
- Hehenberger, Peter (2011). *Computerunterstützte Fertigung: Eine kompakte Einführung*. Berlin, Heidelberg: Springer-Verlag. ISBN: 9783642134753.
- Jaklic, Gasper u. a. (2010). "On Interpolation by Planar Cubic G^2 Pythagorean-Hodograph Spline Curves". In: *Mathematics of Computation*. [pdf].

Quellen IV


Kara, Sami u. a. (2015). "The 22nd CIRP Conference on Life Cycle Engineering Minimization of the Energy Consumption in Motion Planning for Single-robot Tasks". In: *Procedia CIRP* 29, S. 354–359. ISSN: 2212-8271. DOI: http://dx.doi.org/10.1016/j.procir.2015.02.174. URL: http://www.sciencedirect.com/science/article/pii/S2212827115004886.


Marquardt, Erik (2016). *Handlungsfelder. Additive Fertigungsverfahren*. Hrsg. von VDI Verein Deutscher Ingenieure e.V. Fachbereich Produktionstechnik und Fertigungsverfahren. URL: https://www.vdi.de/fileadmin/vdi_de/redakteur_dateien/gpl_dateien/6242_PUB_GPL_Handlungsfelder_-_Additive_Fertigungsverfahren_Internet.pdf (besucht am 13. 08. 2016).


Piegl, Les und Wayne Tiller (1997). *The NURBS Book*. 2. Aufl. New York, NY, USA: Springer-Verlag New York, Inc. ISBN: 3-540-61545-8.

Quellen V


R. Malaka A. Butz, H. Hussman (2009). *Medieninformatik*. Pearson Studium.


Russell, David u. a. (2007). *Apparatus and Methods for 3D Printing*. United States Patent, Patent N0.: US 7,291,002 B2.


Sencer, Burak und Eiji Shamoto (2014). "Curvature-continuous sharp corner smoothing scheme for Cartesian motion systems". In: *2014 IEEE 13th International Workshop on Advanced Motion Control (AMC)*. [pdf] und [Version 2 - pdf]. Piscataway, NJ: IEEE, S. 374–379. ISBN: 978-1-4799-2323-6. DOI: \url{10.1109/AMC.2014.6823311}.


Sindermann, Sebastian (2014). *Schnittstellen und Datenaustauschformate*. German. Springer Berlin Heidelberg, S. 327–347. ISBN: 978-3-662-43815-2. DOI: 10.1007/978-3-662-43816-9_14. URL: http://dx.doi.org/10.1007/978-3-662-43816-9_14.

Quellen VI

- Tajima, Shingo und Burak Sencer (2016). "Kinematic corner smoothing for high speed machine tools". In: *International Journal of Machine Tools and Manufacture* 108. [pdf], S. 27–43. ISSN: 08906955. DOI: \url{10.1016/j.ijmachtools.2016.05.009}.
- — (2017). "Global tool-path smoothing for CNC machine tools with uninterrupted acceleration". In: *International Journal of Machine Tools and Manufacture* 121. [pdf], S. 81–95. ISSN: 08906955. DOI: \url{10.1016/j.ijmachtools.2017.03.002}.
- Waterloo Maple Inc. 2017 (2017a). *Description*. letzter Zugriff 14.09.2017. URL: https://de.maplesoft.com/support/help/Maple/view.aspx?path=module/description.
- — (2017b). *Export*. letzter Zugriff 14.09.2017. URL: https://de.maplesoft.com/support/help/Maple/view.aspx?path=module/export.

Quellen VII


Waterloo Maple Inc. 2017 (2017c). *Module*. letzter Zugriff 14.09.2017. URL: https://de.maplesoft.com/support/help/maple/view.aspx?path=module.


— (2017d). *ModuleLoad*. letzter Zugriff 14.09.2017. URL: https://de.maplesoft.com/support/help/Maple/view.aspx?path=ModuleLoad.


— (2017e). *Option*. letzter Zugriff 14.09.2017. URL: https://de.maplesoft.com/support/help/Maple/view.aspx?path=module/option.


— (2017f). *Procedures*. letzter Zugriff 14.09.2017. URL: https://de.maplesoft.com/support/help/Maple/view.aspx?path=procedure.

Quellen VIII

 Zafeiropoulos, Anastasios u. a. (2020). "Benchmarking and Profiling 5G Verticals' Applications: An Industrial IoT Use Case". In: *IEEE Conference on Network Softwarization, NetSoft 2020*. DOI: 10.1109/NetSoft48620.2020.9165393.

 Zeidler, E., Hrsg. (2013). *Springer-Taschenbuch der Mathematik*. 3., neu bearb eitete und erweiterte Auflage. Springer Verlag. ISBN: 978-3-8348-2359-5.